

电子设计基础训练

——Arduino模拟I/O口实验

北京航空航天大学
电子信息工程学院





Arduino模拟I/O口实验

- 模拟信号与数字信号
- 模数(AD)-数模(DA)转换
- 实验1: 电位器模拟输入
- 实验2: PWM模拟输出
- 实验3: 呼吸灯电路
- 实验4: 亮度可控LED灯
- 实验5: 颜色可调三色灯



连续离散-模拟数字

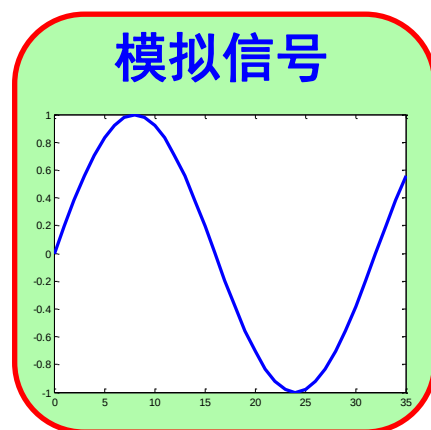
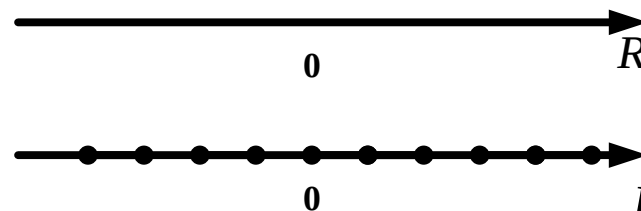
■ 信号分类 $y=f(x)$ 连续、离散

➤ 自变量为连续值的信号称为连续信号

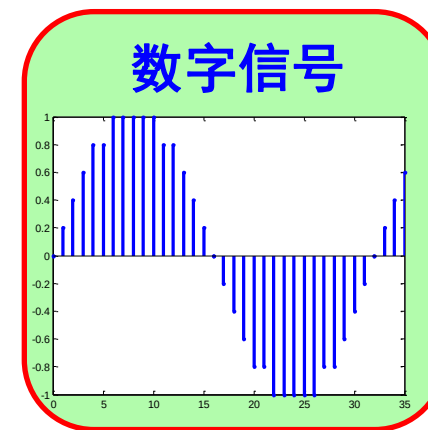
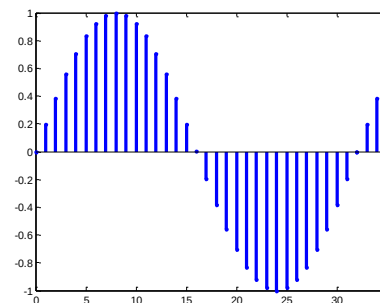
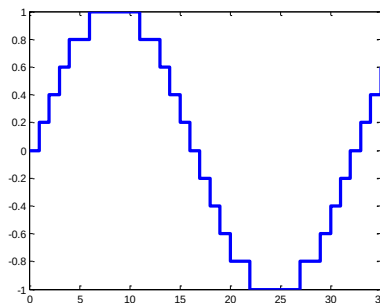
➤ 自变量为离散值的信号称为离散信号

➤ 自变量和函数值均为连续值的信号称为模拟信号

➤ 自变量和函数值均为离散值的信号称为数字信号



连续信号

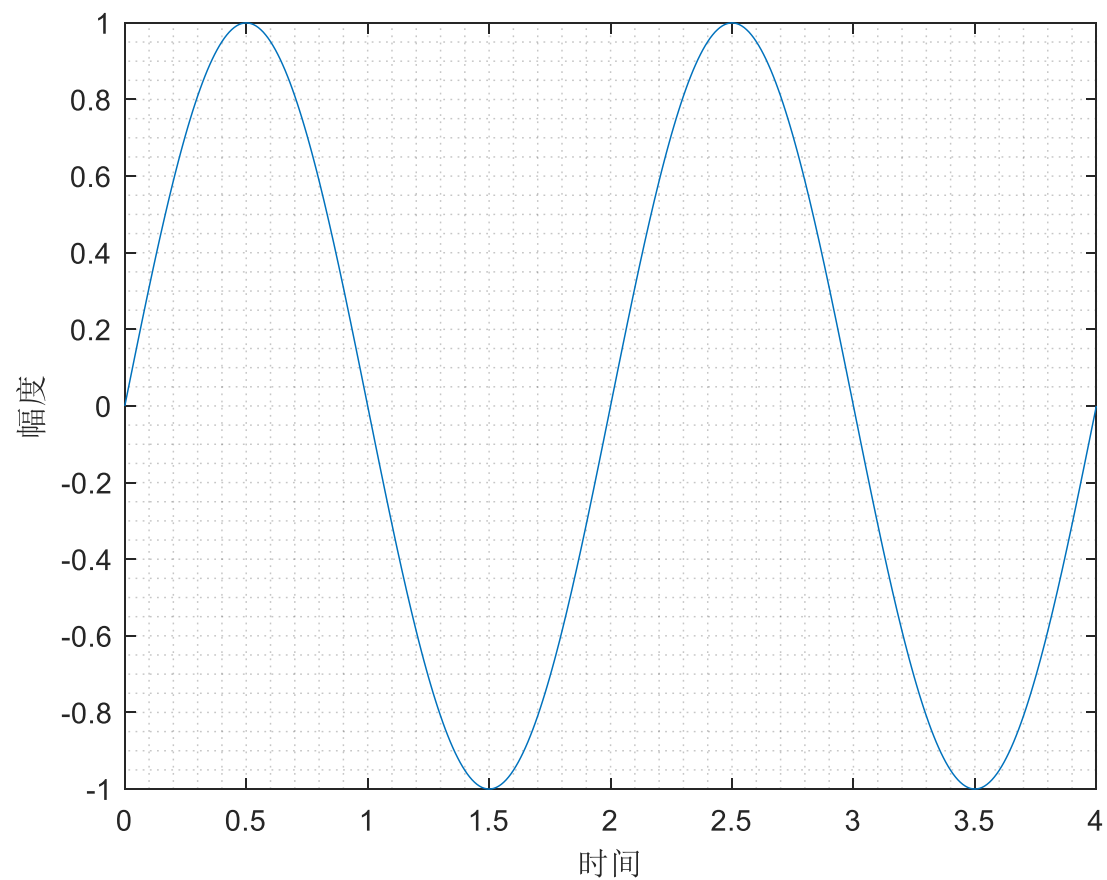


离散信号

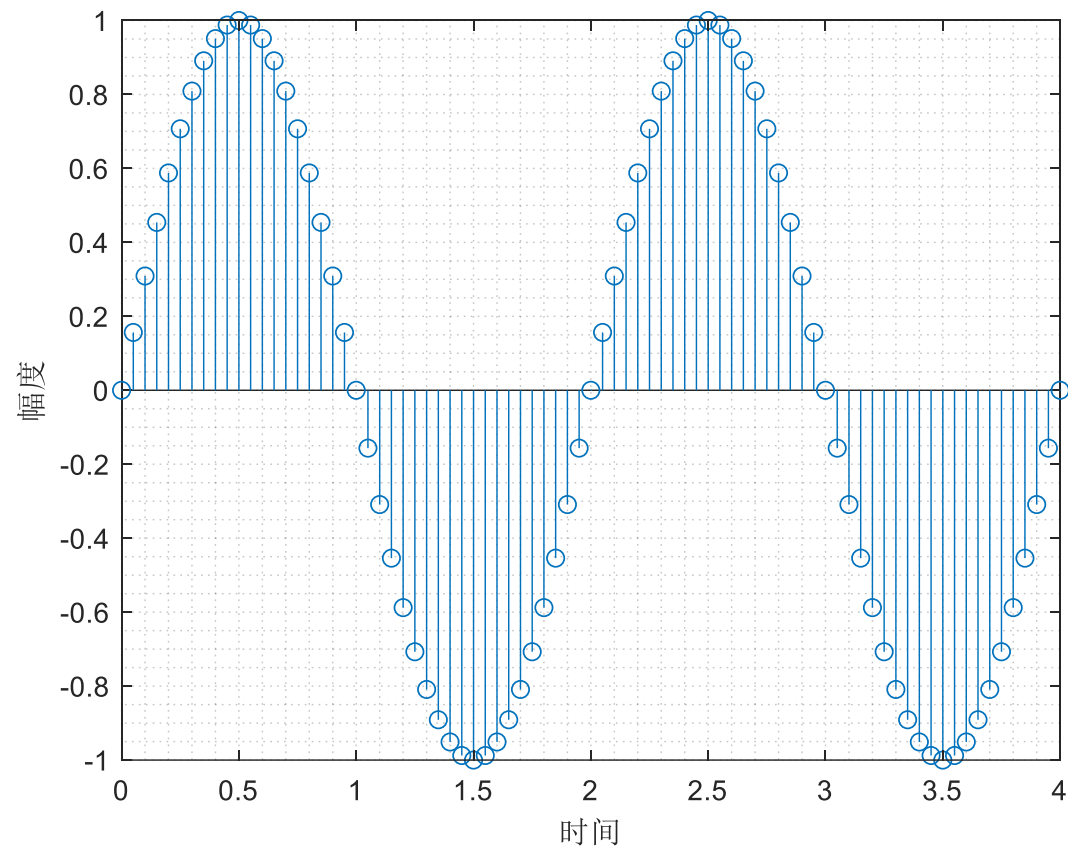


模拟信号与数字信号

模拟信号：时间和幅值都连续



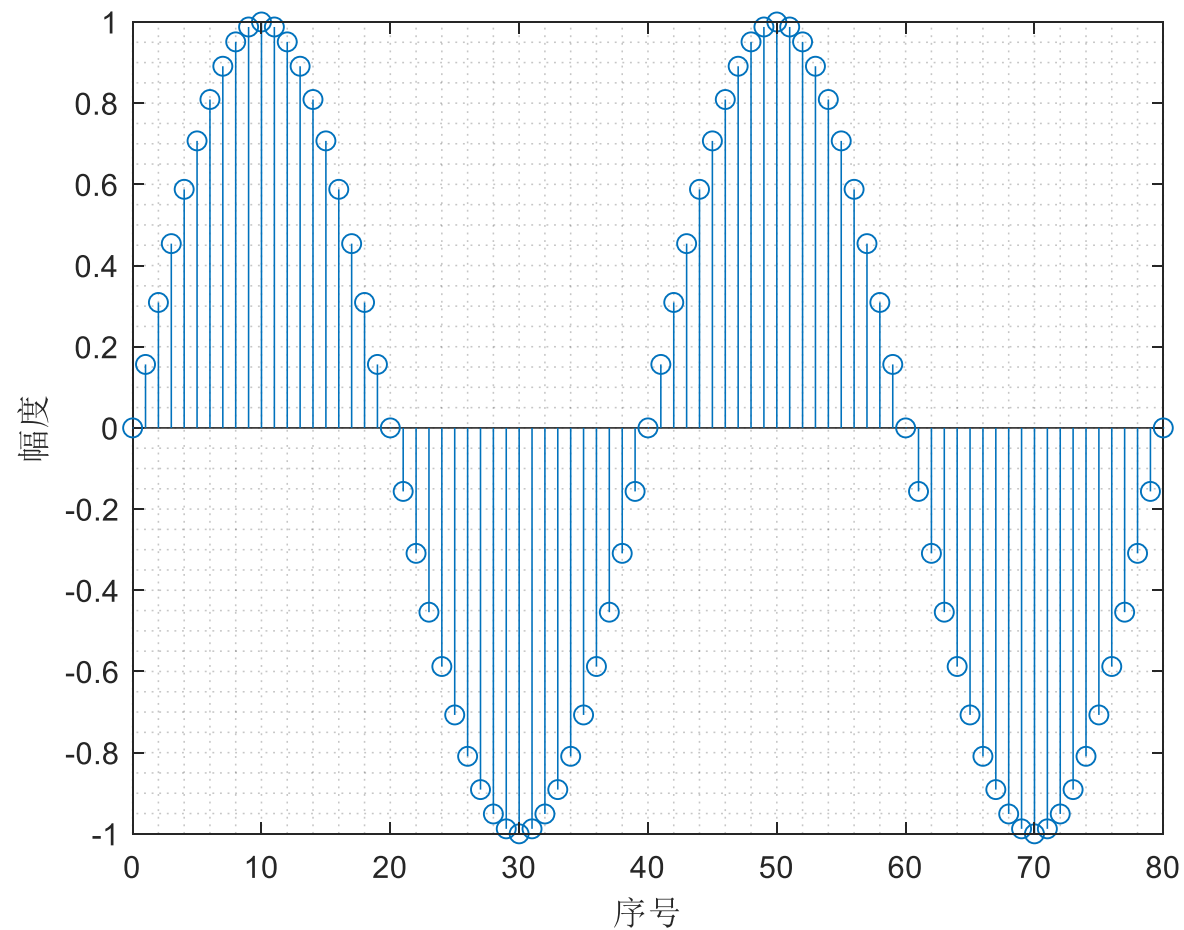
离散时间信号：时间离散、幅值连续



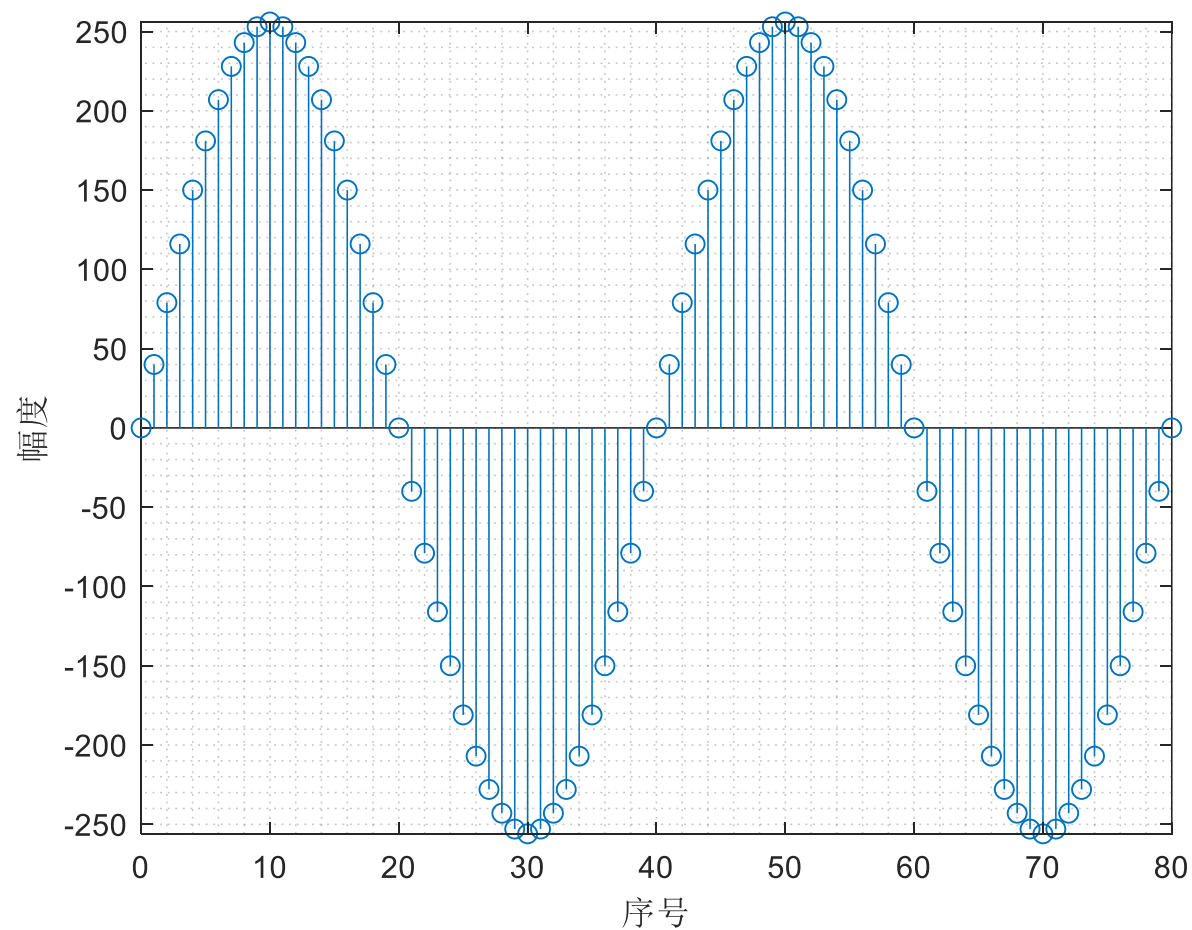


模拟信号与数字信号

离散时间信号：时间离散、幅值连续



数字信号：时间和幅值都离散

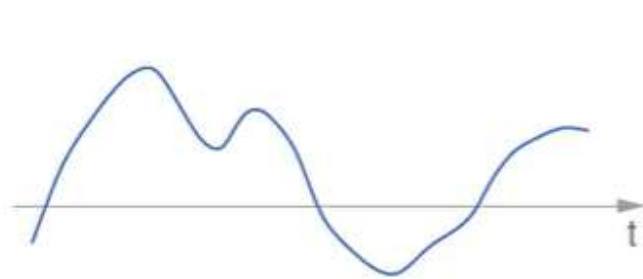




模拟信号与数字信号

■ 模拟信号、电路

- 信号形式：**在时间上和数值上均是连续的。**如温度、湿度、压力、长度、电流等等
- 表示方式：模拟自然信号、用连续的电压信号表示信息
- 分析方法：如何不失真的信号传输、处理

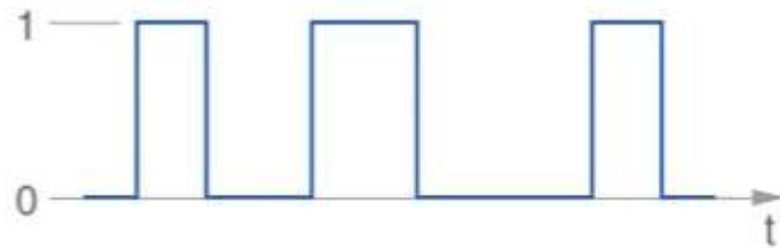


analog signal



■ 数字信号、电路

- 信号形式：**在时间上和数值上均是离散的。**
- 表示方式：**采样、量化**用离散的电压序列（0/1**编码**）表示信息
- 分析方法：输入和输出信号的逻辑关系



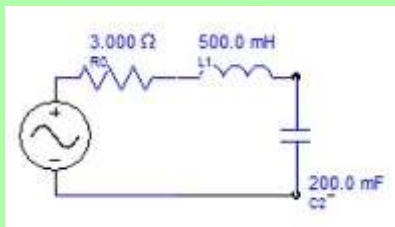
digital signal



连续离散-模拟数字

■ **系统**是对信号进行处理实现某功能的物理设备

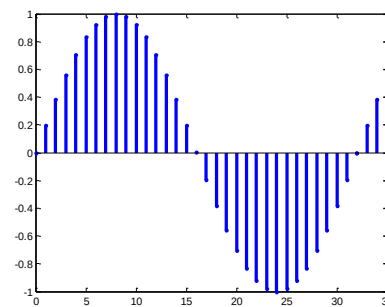
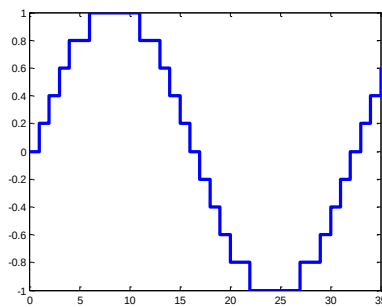
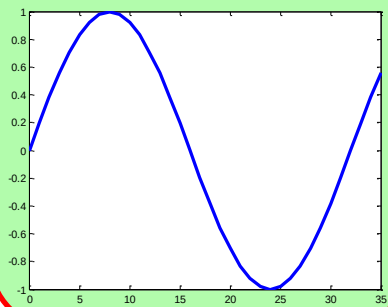
模拟系统



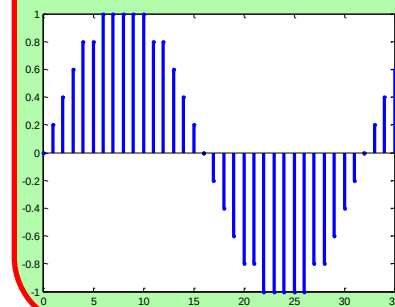
数字系统



模拟信号



数字信号



连续信号

离散信号

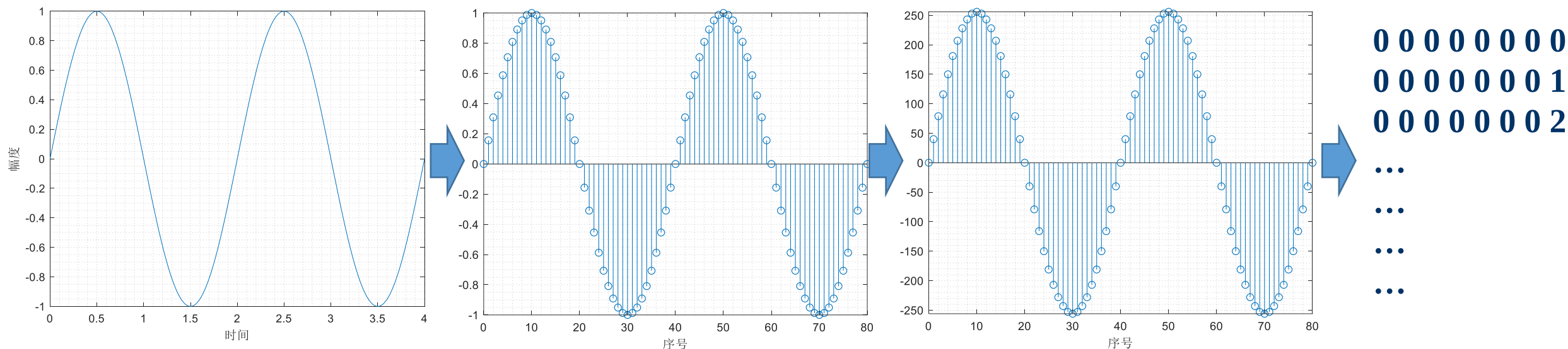


Arduino模拟I/O口实验

- 模拟信号与数字信号
- **模数(AD)-数模(DA)转换**
- **实验1：电位器模拟输入**
- **实验2：PWM模拟输出**
- **实验3：呼吸灯电路**
- **实验4：亮度可控LED灯**
- **实验5：颜色可调三色灯**



模数（Analog to Digital, AD）转换过程



$$x(t) = A \cos(\omega t + \varphi) = A \sin(\pi / 2 - (\omega t + \varphi))$$



奈奎斯特（Nyquist）采样定律

- **采样定律**：采样频率（ f_s ）高于信号中最高频率（ f_{max} ）的两倍时（ $f_s \geq 2f_{max}$ ），采样之后的数字信号完整地保留了原始信号中的信息（可以恢复原始信号）；
- 又称奈奎斯特采样条件， $f_s = 2f_{max}$ 称为**奈奎斯特采样率**；
- 采样频率**大于**奈奎斯特采样率称为**过采样**，采样频率**小于**奈奎斯特采样率称为**欠采样**；
- 注意采样时间间隔为采样频率的倒数 $T_s = 1/f_s$ 。例如，采样频率1kHz,则采样时间间隔为1ms。



奈奎斯特（Nyquist）采样定律





奈奎斯特 (Nyquist) 采样定律





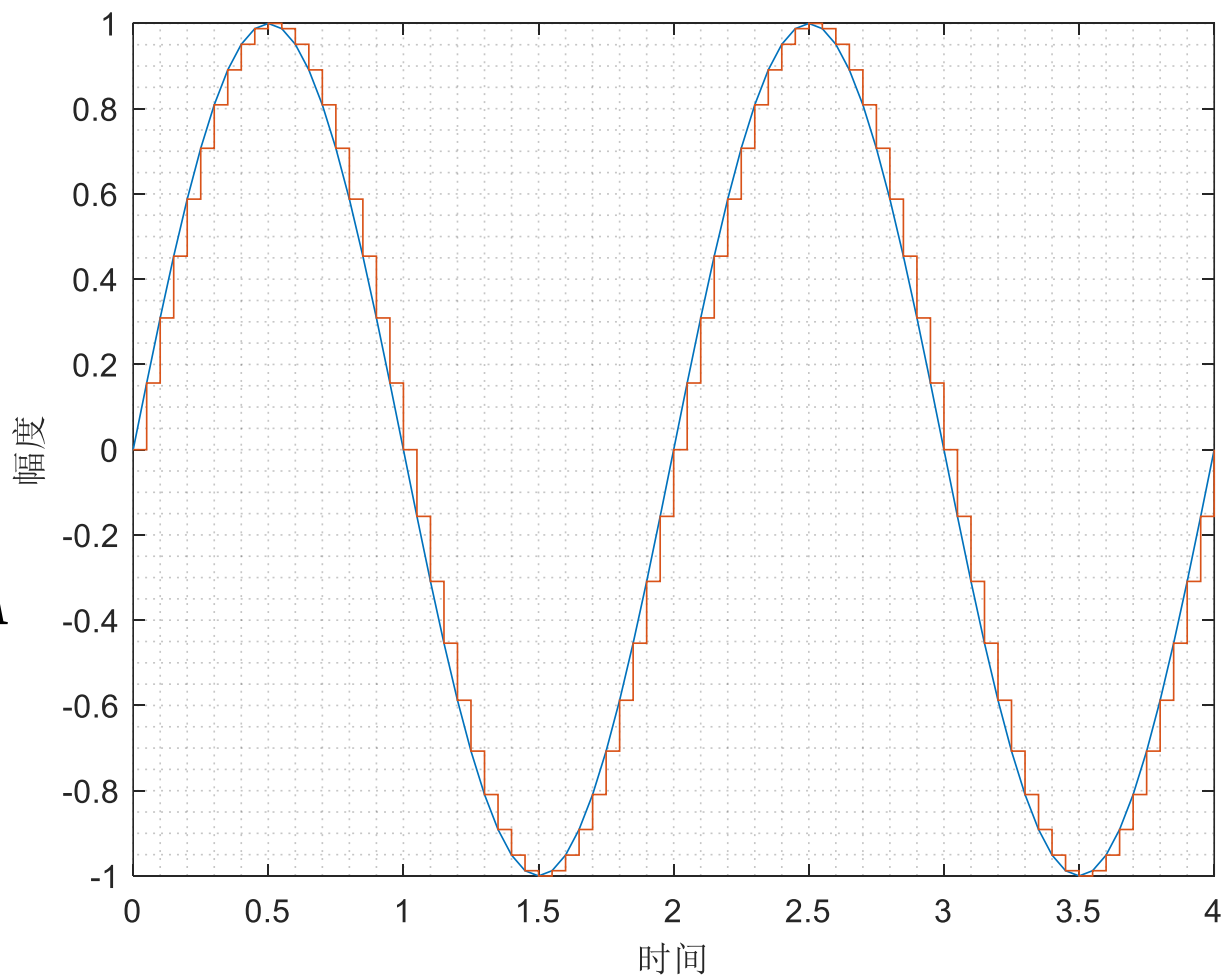
量化：幅度离散化

- 需要指定量化单位 ΔA ，量化位数 M bit。

$$x(t) = \begin{cases} 1, \Delta A \leq x(t) < 2\Delta A \\ 2, 2\Delta A \leq x(t) < 3\Delta A \\ \dots \\ 2^M - 1, (2^M - 1)\Delta A \leq x(t) < 2^M \Delta A \\ 2^M, x(t) \geq 2^M \Delta A \end{cases}$$

饱和

有限量化范围： $0 \sim 2^M \Delta A$





二进制

- 二进制：以2为基数的记数系统，“逢2进1”；
- 1位二进制数称为1bit（比特），32位系统、64位系统；
- 数字信息系统中1byte（字节）为8bit，最大值为 $2^8-1 = 255$ ；
- 1字长为16比特，最大值为 $2^{16}-1 = 65535$ ；
- 考虑正负时，最高位表示正负号：

1 0 0 1 1 1 0 1 → 不考虑正负号 $2^7+2^4+2^3+2^2+2^0 = 157$
考虑正负号 - $(2^4+2^3+2^2+2^0) = -30$

1 1 1 1 1 1 1 1 → 不考虑正负号 $2^7+2^6+2^5+2^4+2^3+2^2+2^1+2^0 = 255$
考虑正负号 - $(2^6+2^5+2^4+2^3+2^2+2^1+2^0) = -127$



十六进制

■ 为便于运算4位二进制→1位十六进制，1字节为2位十六进制

十进制	二进制	十六进制	十进制	二进制	十六进制
0	0 0 0 0	0	8	1 0 0 0	8
1	0 0 0 1	1	9	1 0 0 1	9
2	0 0 1 0	2	10	1 0 1 0	a
3	0 0 1 1	3	11	1 0 1 1	b
4	0 1 0 0	4	12	1 1 0 0	c
5	0 1 0 1	5	13	1 1 0 1	d
6	0 1 1 0	6	14	1 1 1 0	e
7	0 1 1 1	7	15	1 1 1 1	f



编码：生成数据

- 数值→二进制数→存储或传输
- 字符→ASCII码→二进制数→存储或传输

高四位 低四位	ASCII控制字符												ASCII打印字符												
	0000						0001						0010	0011	0100	0101	0110	0111							
	0						1						2	3	4	5	6	7							
	十进制	字符	Ctrl	代码	转义字符	字符解释	十进制	字符	Ctrl	代码	转义字符	字符解释	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	Ctrl
0000	0	0		^@	NUL	\0	空字符	16	▶	^P	DLE		数据链路转义	32		48	0	64	@	80	P	96	`	112	p
0001	1	1	☺	^A	SOH		标题开始	17	◀	^Q	DC1		设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q
0010	2	2	☹	^B	STX		正文开始	18	↕	^R	DC2		设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r
0011	3	3	♥	^C	ETX		正文结束	19	!!	^S	DC3		设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s
0100	4	4	♦	^D	EOT		传输结束	20	¶	^T	DC4		设备控制 4	36	\$	52	4	68	D	84	T	100	d	116	t
0101	5	5	♣	^E	ENQ		查询	21	§	^U	NAX		否定应答	37	%	53	5	69	E	85	U	101	e	117	u
0110	6	6	♠	^F	ACK		肯定应答	22	—	^V	SYN		同步空闲	38	&	54	6	70	F	86	V	102	f	118	v
0111	7	7	•	^G	BEL	la	响铃	23	↕	^W	ETB		传输块结束	39	'	55	7	71	G	87	W	103	g	119	w
1000	8	8	▣	^H	BS	lb	退格	24	↑	^X	CAN		取消	40	(	56	8	72	H	88	X	104	h	120	x
1001	9	9	○	^I	HT	lt	横向制表	25	↓	^Y	EM		介质结束	41	)	57	9	73	I	89	Y	105	i	121	y
1010	A	10	◎	^J	LF	ln	换行	26	→	^Z	SUB		替代	42	*	58	:	74	J	90	Z	106	j	122	z



模数转换器 (Analog to Digital Converter, ADC)

■ 实现模拟信号到数字信号的转变

- 并联比较型，逐次逼近型，连续时间型、双积分型.....
- 结构简单、转换速率快
- 并联比较型由等值电阻、电压比较器、D型锁存器和优先编码器构成



TLC5510并联比较型ADC

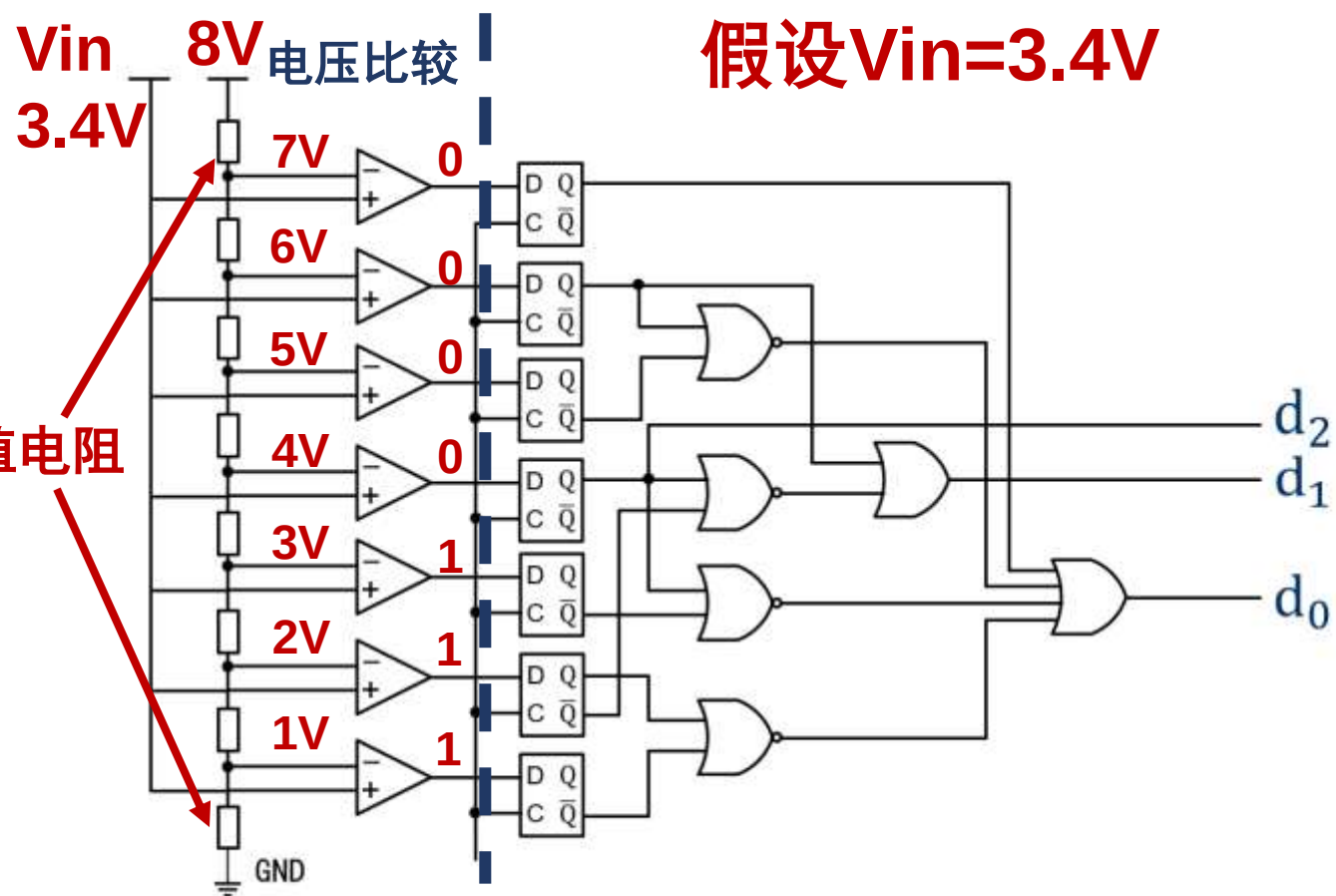


国产RS1430B ADC



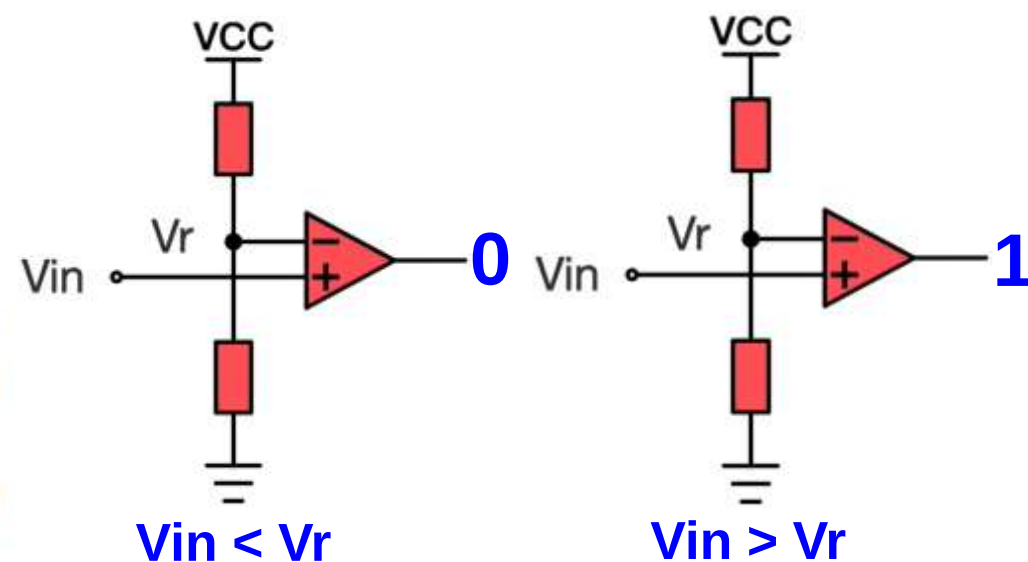
模数转换器 (ADC)

并联比较型由等值电阻、电压比较器、D型锁存器和优先编码器构成



■ 电压比较器用于比较两个电压的大小

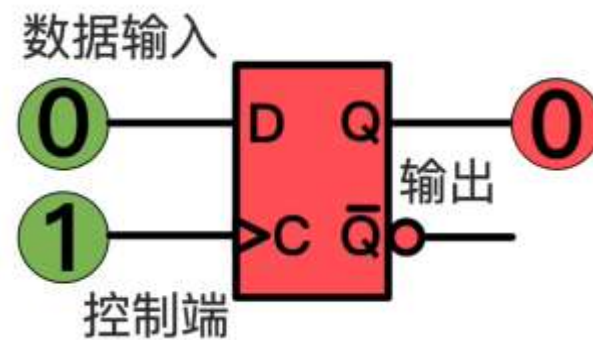
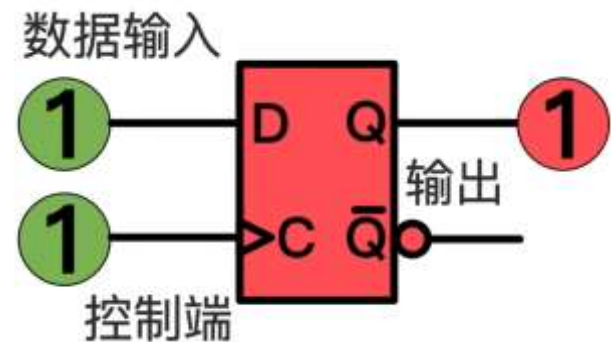
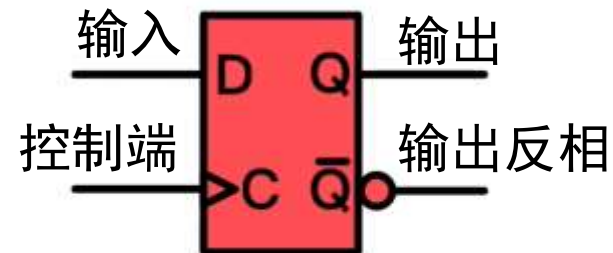
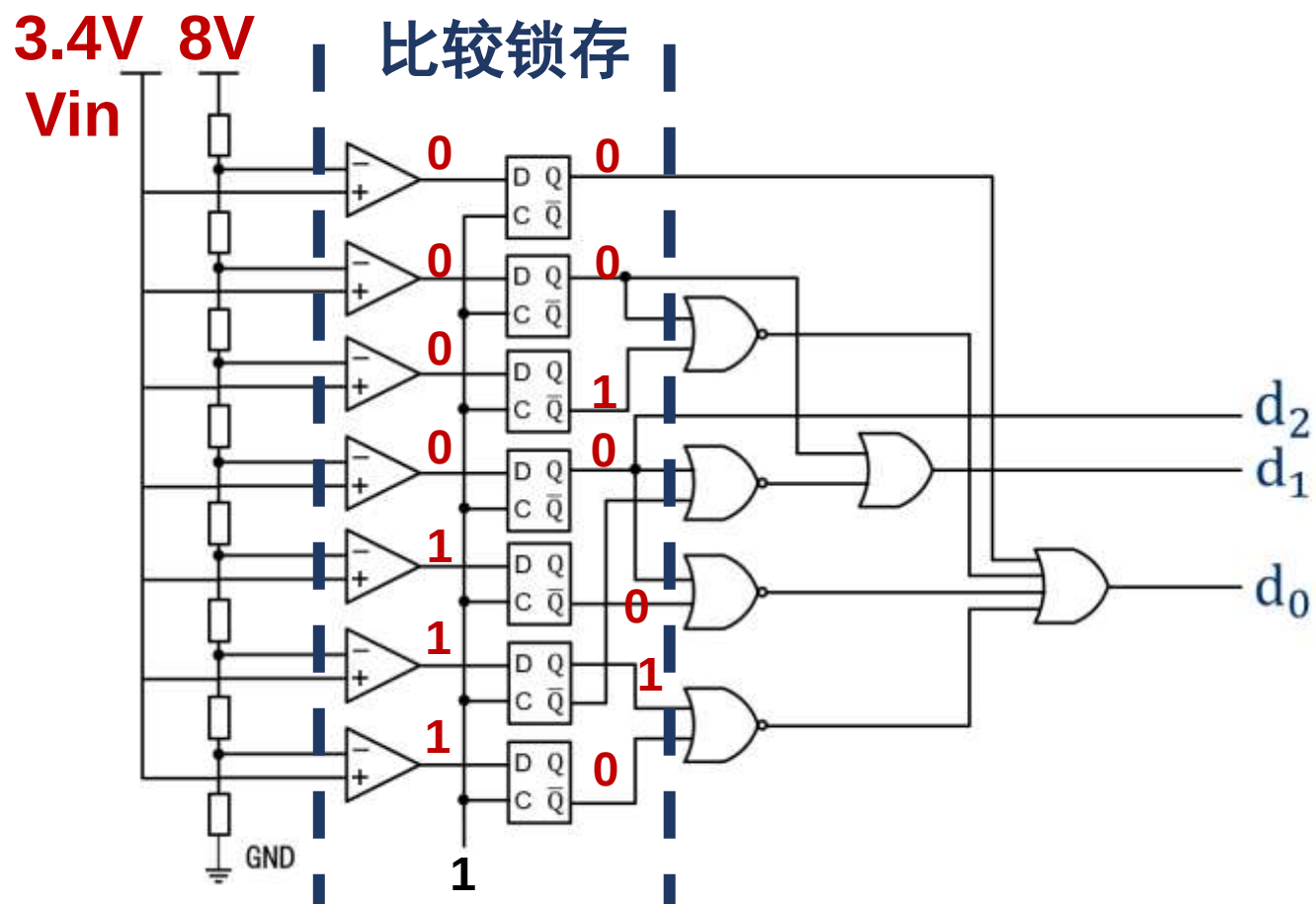
➤ 输入电压 V_{in} 、量化电压 V_r





模数转换器 (ADC)

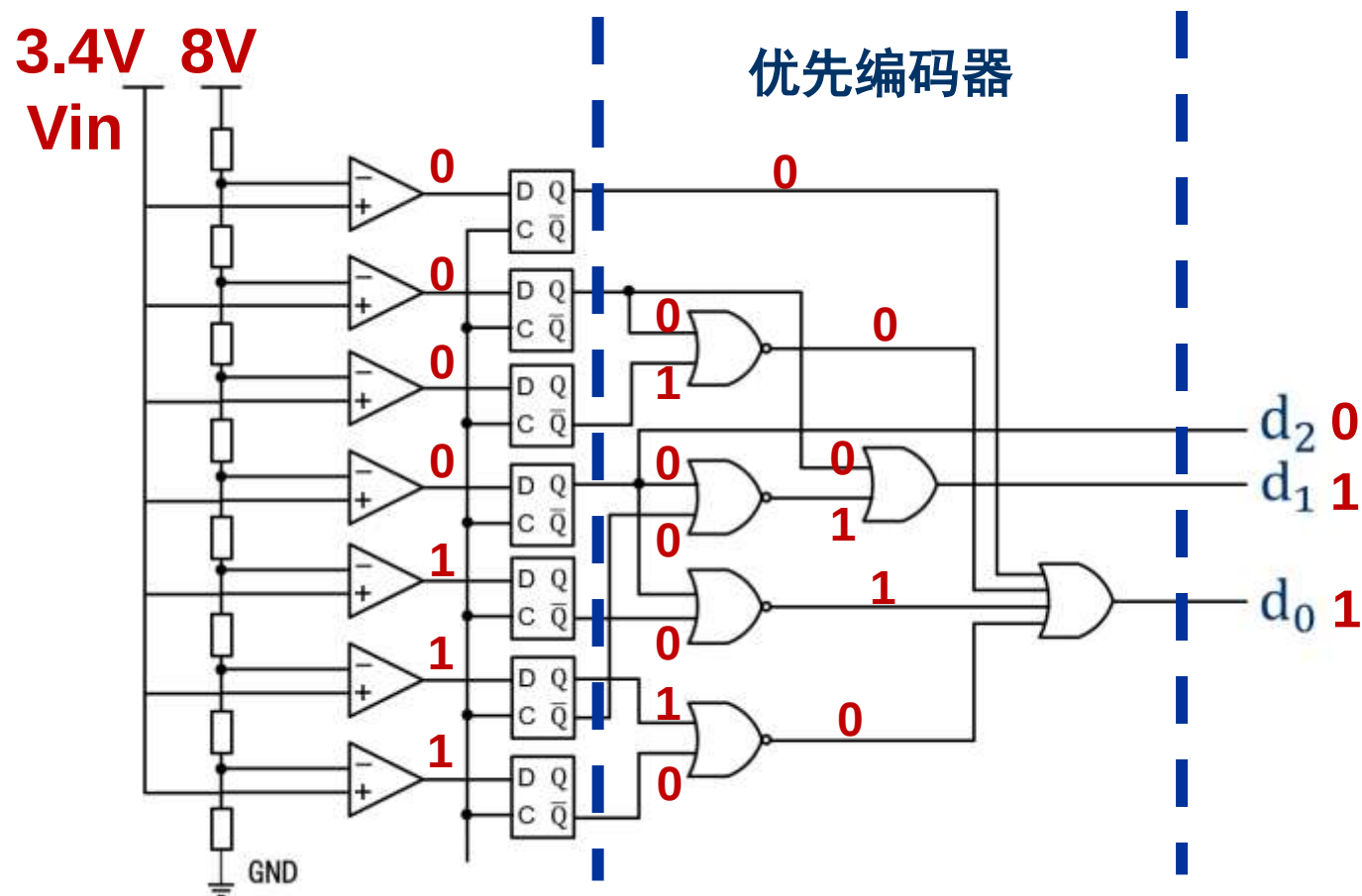
■ D型触发器/D型锁存器可以存储一位数据





模数转换器（ADC）：逻辑门电路

■ 利用逻辑门电路实现特定的逻辑关系



$$C=A\&B$$



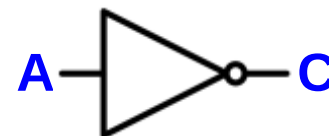
与门 (AB都为1)

$$\text{或门 } C=A\mid B$$



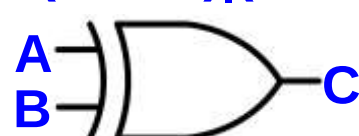
与门 (AB其一为1)

$$C=\sim A$$



非门 (A取反)

$$C=(\sim A\&B)\mid(A\&\sim B)$$



异或门 (AB不同)

模拟电压	比较器输出	二进制值
0-1V	000 0000	000
1-2V	000 0001	001
2-3V	000 0011	010
3-4V	000 0111	011
4-5V	000 1111	100
5-6V	001 1111	101
6-7V	011 1111	110
7-8V	111 1111	111



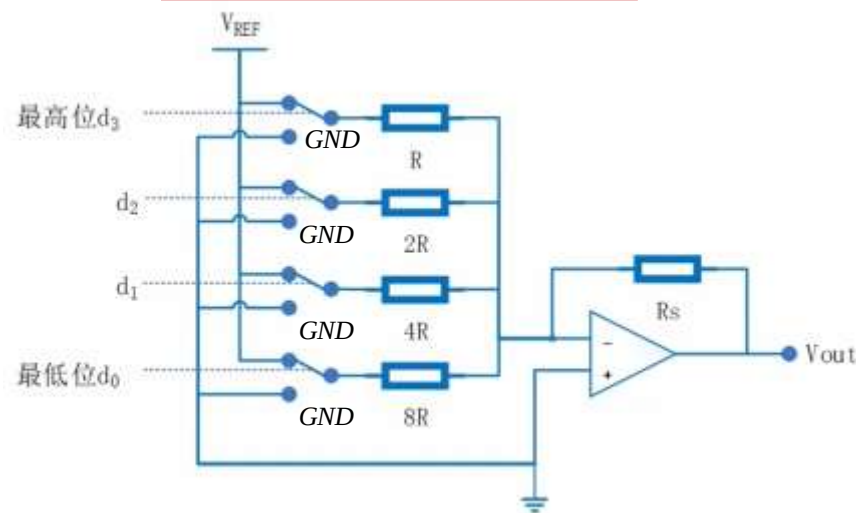
数模转换器（DAC）：模拟电路

■ 数字信号转换成模拟信号

- 4位二进制输入，对应16个量化电平，
不同位二进制数对电压的贡献不同
- **如0001 $\leftrightarrow$ Vref/16, 0100 $\leftrightarrow$ 4*Vref/16**
- 利用开关、电阻和运算放大器，即可实现数字信号0和1（开关的打开和闭合），到模拟电压输出的转换



d_3	d_2	d_1	d_0
$\frac{8}{16}V_{ref}$	$\frac{4}{16}V_{ref}$	$\frac{2}{16}V_{ref}$	$\frac{1}{16}V_{ref}$





Arduino模拟I/O口实验

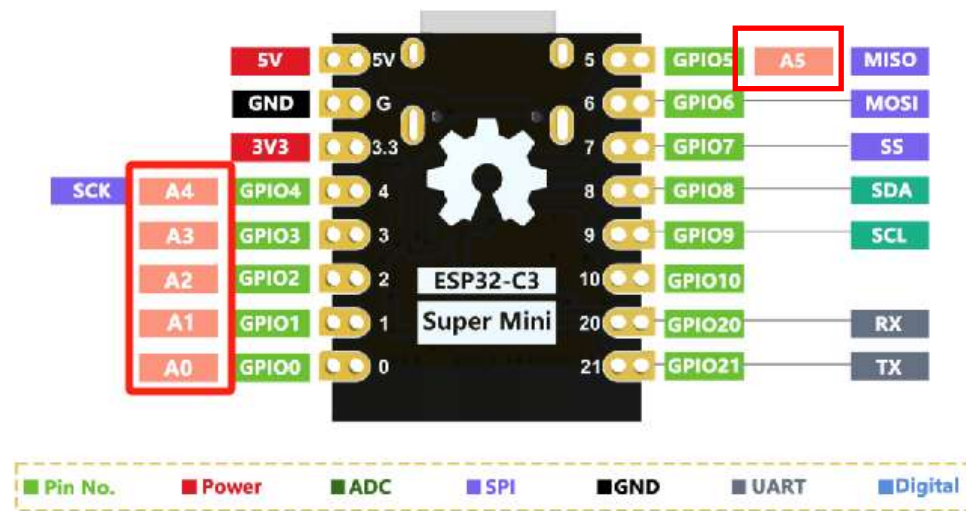
- 模拟信号与数字信号
- 模数(AD)-数模(DA)转换
- **实验1：电位器模拟输入**
- **实验2：PWM模拟输出**
- **实验3：呼吸灯电路**
- **实验4：亮度可控LED灯**
- **实验5：颜色可调三色灯**



实验1：电位器模拟输入

■ ESP32C3 supermini模拟IO的输入

- 6个模拟IO输入引脚：A0—A5
- ESP32单片机：A/D转换器
- 将外部输入的模拟信号转换为芯片运算时可以识别的数字信号，从而实现读入模拟值的功能
- 12位分辨率，采样值0-4095
- 输入电压范围：0-3.3V
- 也可用做GPIO





实验1：电位器模拟输入

■ESP32开发板的模数转换

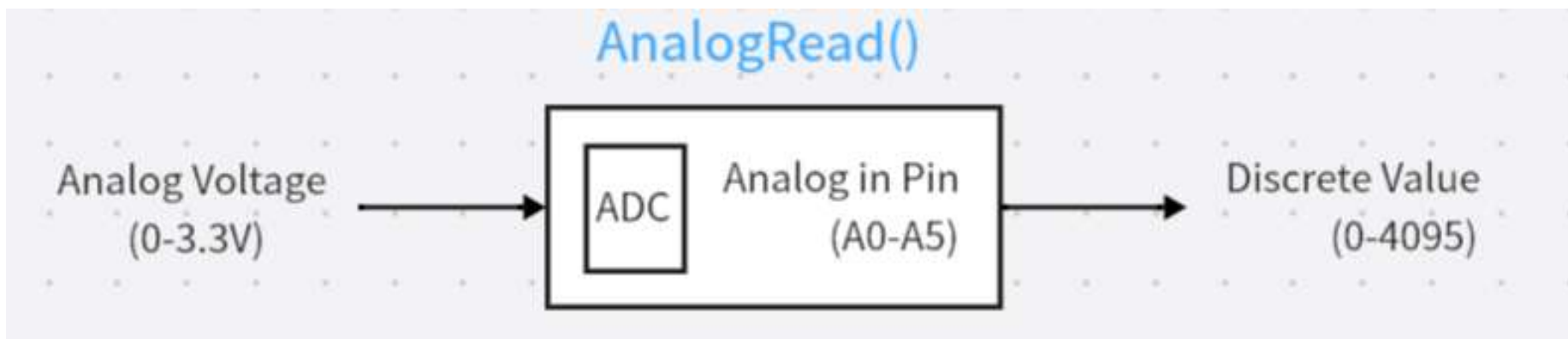
- Value = analogRead(Pin)
- 将模拟电压（**0-3.3V**）量化转换为12位采样值（**0-4095**）

模拟电压V 量化值Value

1.0V 1291

1.5V 1861

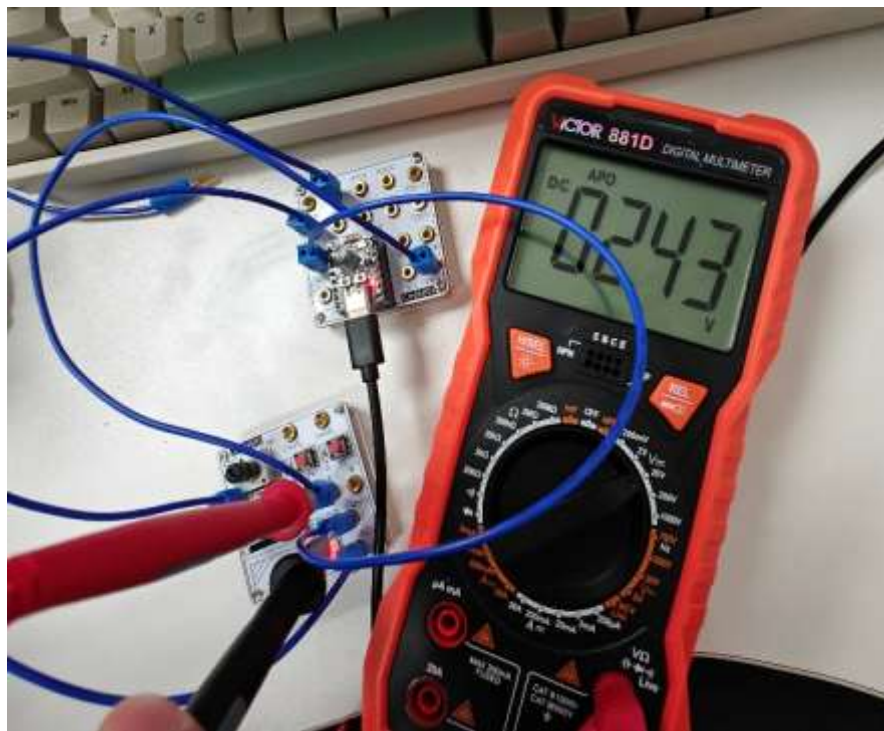
2.0V 2482





实验1：电位器模拟输入

- 利用ESP32开发板读取电位器的模拟电压。
- 将读取到的量化值转换为模拟电压，利用串口输出到串口监视器，用万用表测量实际电压，比较差异。

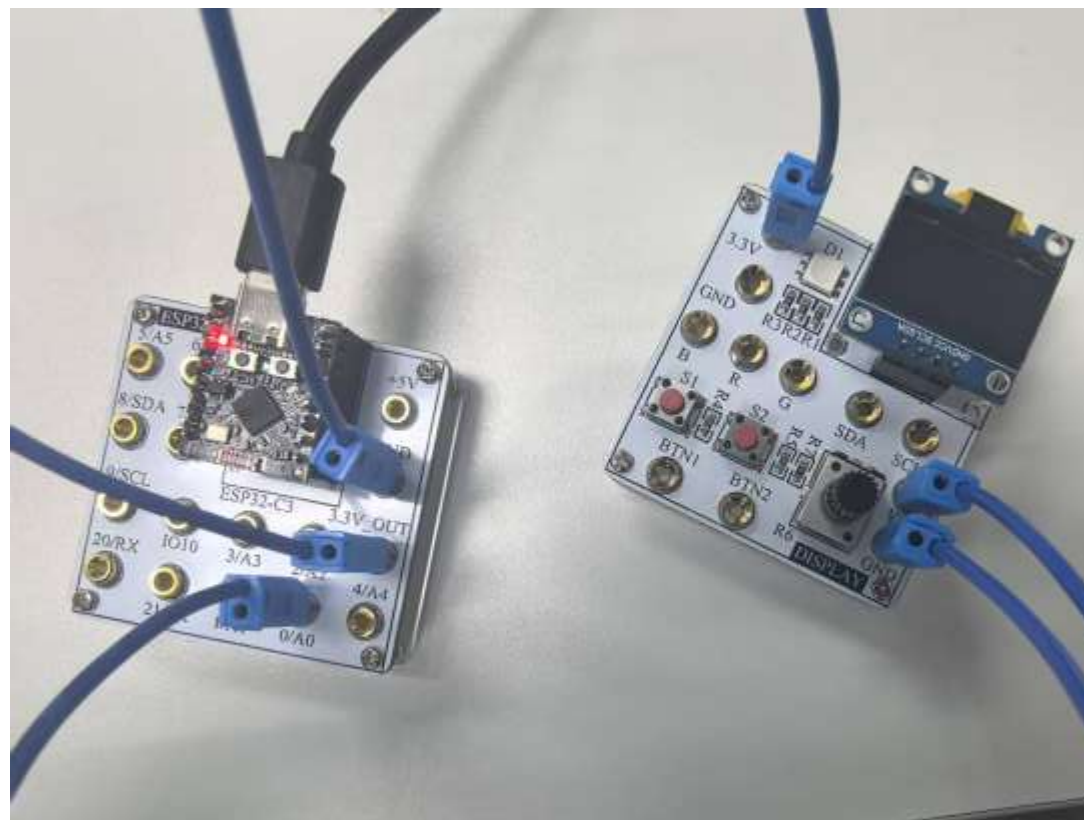




实验1：实物连线

■ 电位器

- 连接两个板子的3.3V脚
- 连接两个板子的接地GND
- Ver脚接ESP32模拟输入引脚0
- 电位器端的电压是一个模拟信号
- 阻值连续变化，电压也连续变化





实验1：实验代码

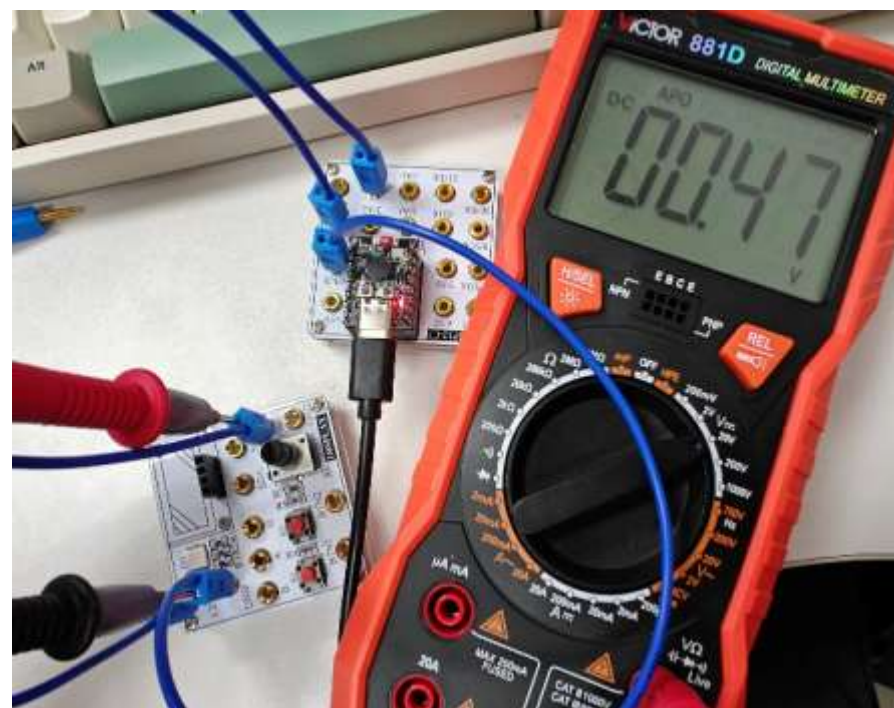
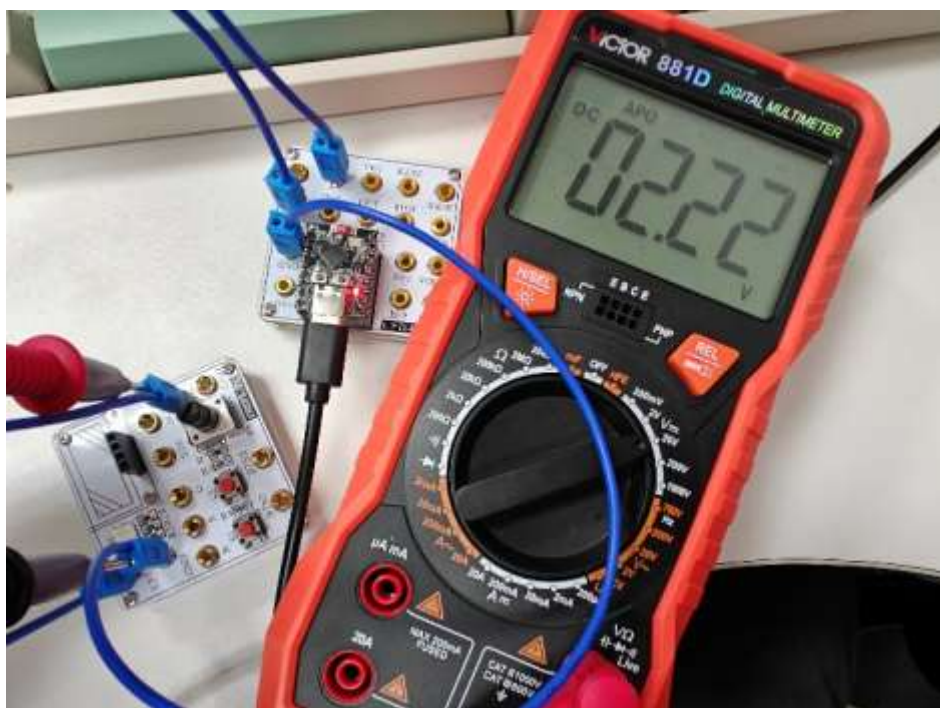
- **Serial.begin(baudrate)** 设置串口通信波特率
- **analogRead(pin)** 读（输入）引脚
- **Serial.print() / Serial.println()** 串口输出内容

```
1  #define pots 0
2  void setup() {
3      Serial.begin(9600);
4  }
5
6  void loop() {
7      int sensorValue = analogRead(pots);
8      float voltage = sensorValue * 3.3 / 4095.0;
9      Serial.print("电位器电压 = ");
10     Serial.println(voltage);
11     delay(100);
12 }
```




实验1：结果分析

- 利用万用表，记录10组电位器两端电压和串口输出
- 换算后两者结果比较，结果是否一样？为什么？





Arduino模拟I/O口实验

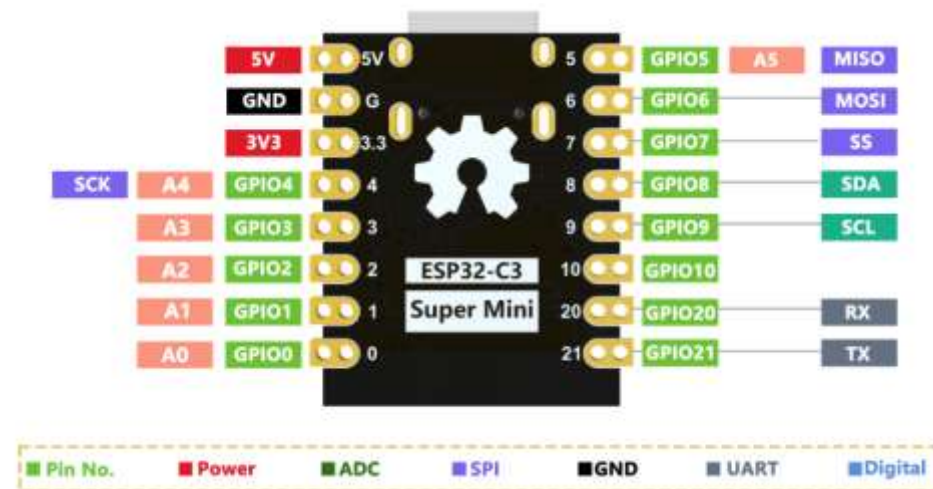
- 模拟信号与数字信号
- 模数(AD)-数模(DA)转换
- **实验1：电位器模拟输入**
- **实验2：PWM模拟输出**
- **实验3：呼吸灯电路**
- **实验4：亮度可控LED灯**
- **实验5：颜色可调三色灯**



实验2：PWM模拟输出

■ ESP32C3 supermini模拟IO的输出

- 没有专门的DAC，不能生成真正的模拟电压
- 可以用11个数字IO接口通过脉冲宽度调制（PWM）模拟一个平均电压值，实现模拟IO的输出
- 输出电压范围：0-3.3V

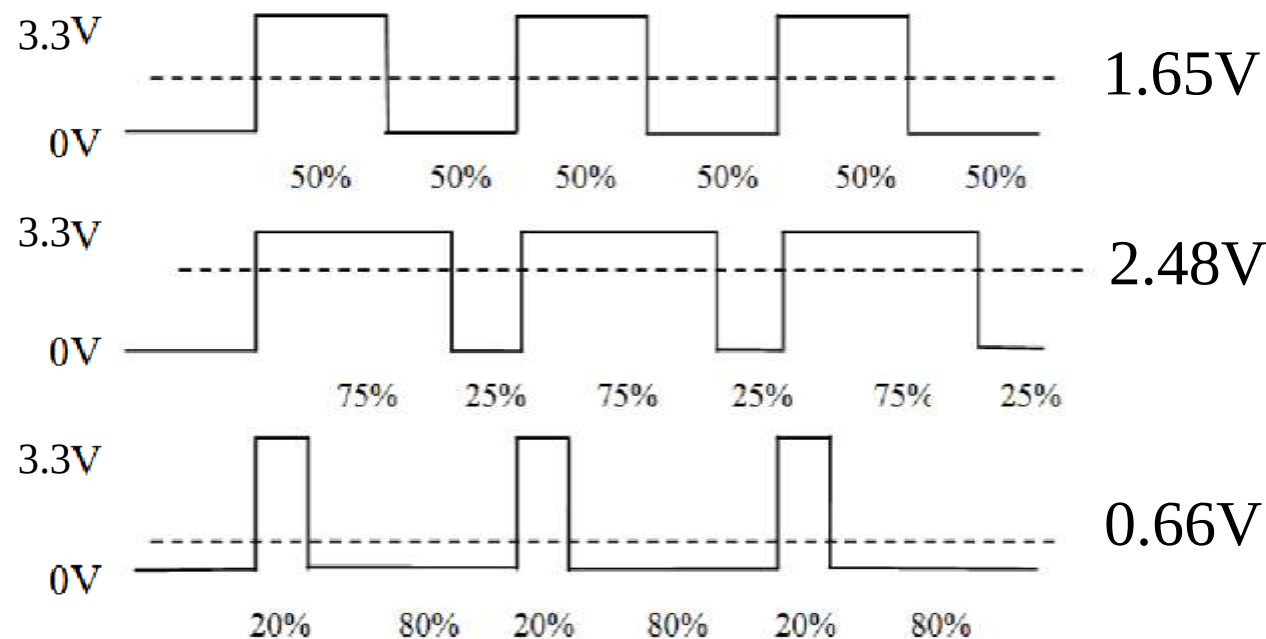




实验2：PWM模拟输出

■ 脉冲宽度调制 (Pulse Width Modulation, PWM)

- 占空比：是指一个周期内高电平所占的比例
- 通过改变方波信号的占空比，可以输出一个近似的模拟电压值，实现数模转换的效果
- 用途：控制LED灯的亮暗渐变、调节电机转速、控制蜂鸣器发声等等





实验2：PWM模拟输出

■ ESP32开发板的数模转换

- PWM输出
- `analogWrite(Pin, Value)`
- 将数字信号（**0-255**）转化为模拟电压（**0-3.3V**）

量化值Value 模拟电压V

100 1.29V

150  1.94V

200 2.59V

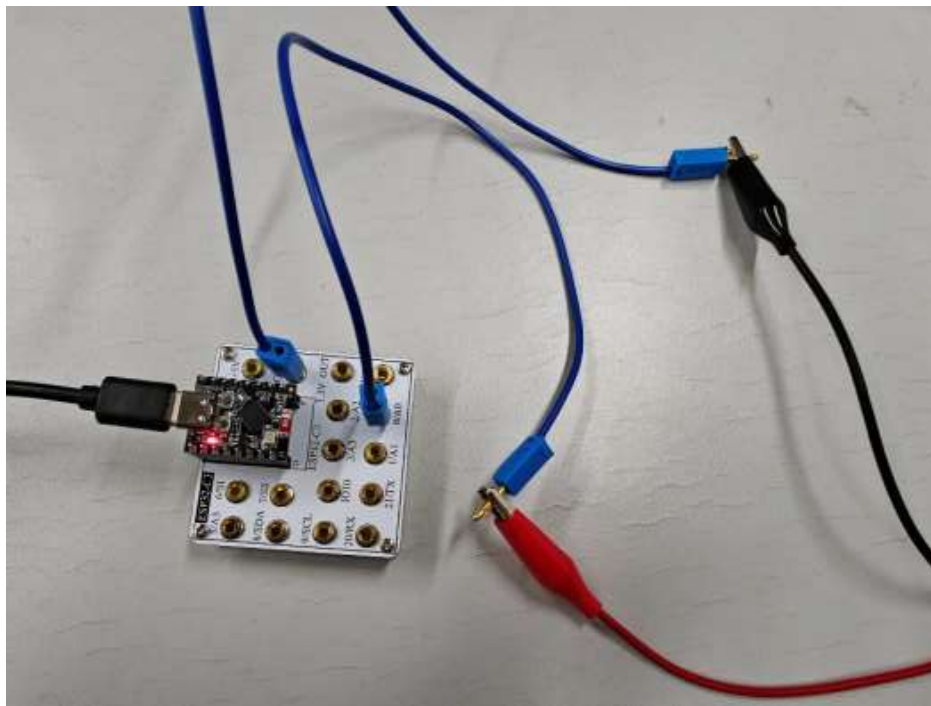
是**analogRead**的逆过程
但是二者量化精度不同





实验2：PWM模拟输出

- 利用串口发送0-255范围内的量化值。
- 利用analogWrite()控制模拟端口输出相应的PWM波，并利用示波器观察PWM波的占空比，记录10组，计算等效电压是否一致

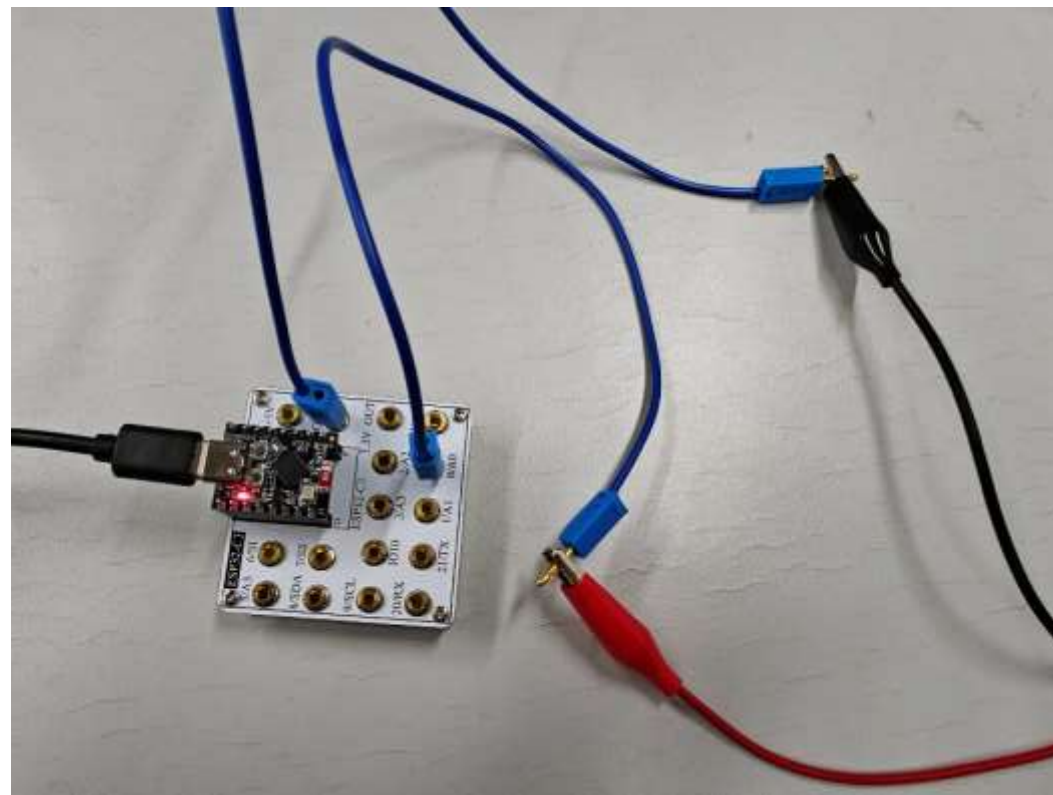




实验2：实物连线

■ PWM模拟输出

- 开发板0号连接示波器BNC线红端
- 开发板接地GND连接示波器BNC线黑端
- BNC线另一端连接示波器





实验2：实验代码

- **Serial.begin(baudrate)**
设置串口通信波特率
- **analogWrite(pin, value)**
写（输出）引脚
- **Serial.print()**
串口输出内容
- **Serial.println()**
串口输出内容（加换行）

```
1  #define PWM_pin 0
2  void setup() {
3      Serial.begin(9600);
4  }
5
6  void loop() {
7      // 检查串口是否有数据
8      if (Serial.available() > 0) {
9          // 读取输入的字符串
10         String input = Serial.readStringUntil('\n');
11
12         // 尝试将字符串转换为整数
13         int value = input.toInt();
14
15         // 检查转换后的值是否在0到255之间
16         if (value >= 0 && value <= 255) {
17             Serial.print("有效值:");
18             Serial.println(value);
19
20             analogWrite(PWM_pin, value);
21         } else {
22             Serial.println("请输入0-255的整数值");
23         }
24     }
25 }
```



实验2：结果分析

- 根据示波器PWM占空比计算等效电压，与输入的量化电平所得等效电压对比，是否一致？



$$\frac{100}{255} \times 3.3 \text{ V} = 1.29 \text{ V}$$



正脉宽392.0us 负脉宽608.0us

$$\frac{392}{392 + 608} \times 3.3 \text{ V} = 1.29 \text{ V}$$



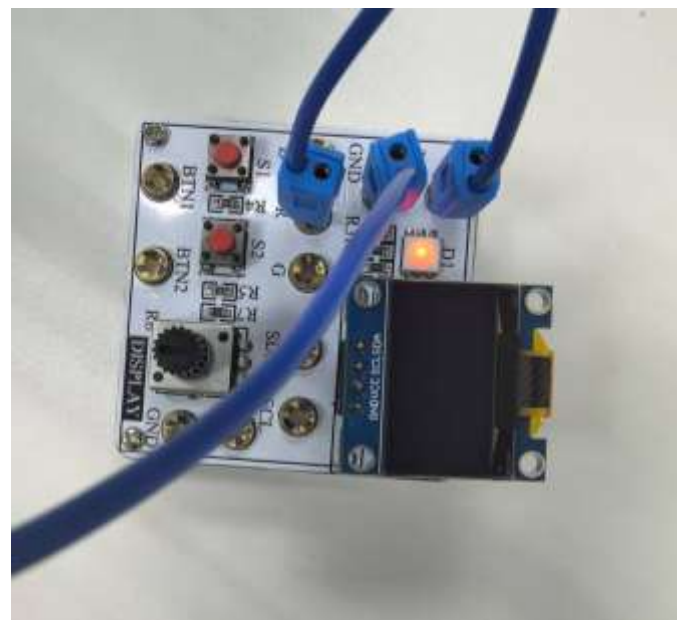
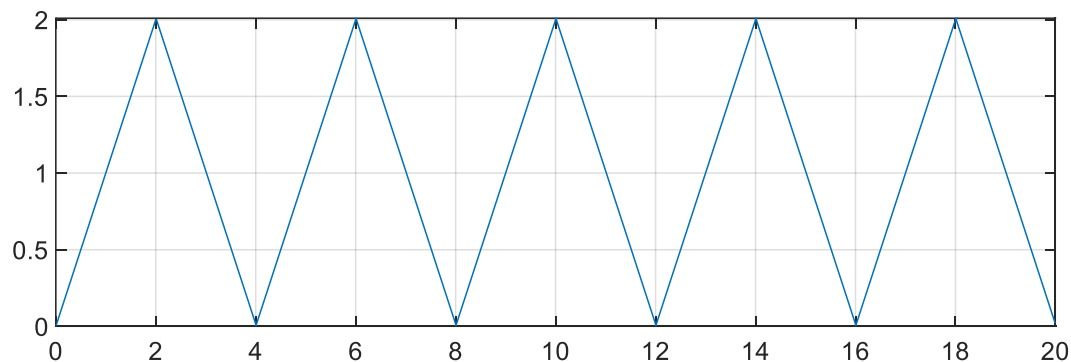
Arduino模拟I/O口实验

- 模拟信号与数字信号
- 模数(AD)-数模(DA)转换
- 实验1： 电位器模拟输入
- 实验2： PWM模拟输出
- **实验3： 呼吸灯电路**
- **实验4： 亮度可控LED灯**
- **实验5： 颜色可调三色灯**



实验3：呼吸灯电路

- ESP32板为典型数字处理系统，利用其生成一个信号幅度随时间变化的（模拟）信号。
- 利用PWM（Pulse-Width Modulation）输出口，输出时变电压使得LED灯以不同的频率进行“呼吸”闪烁。

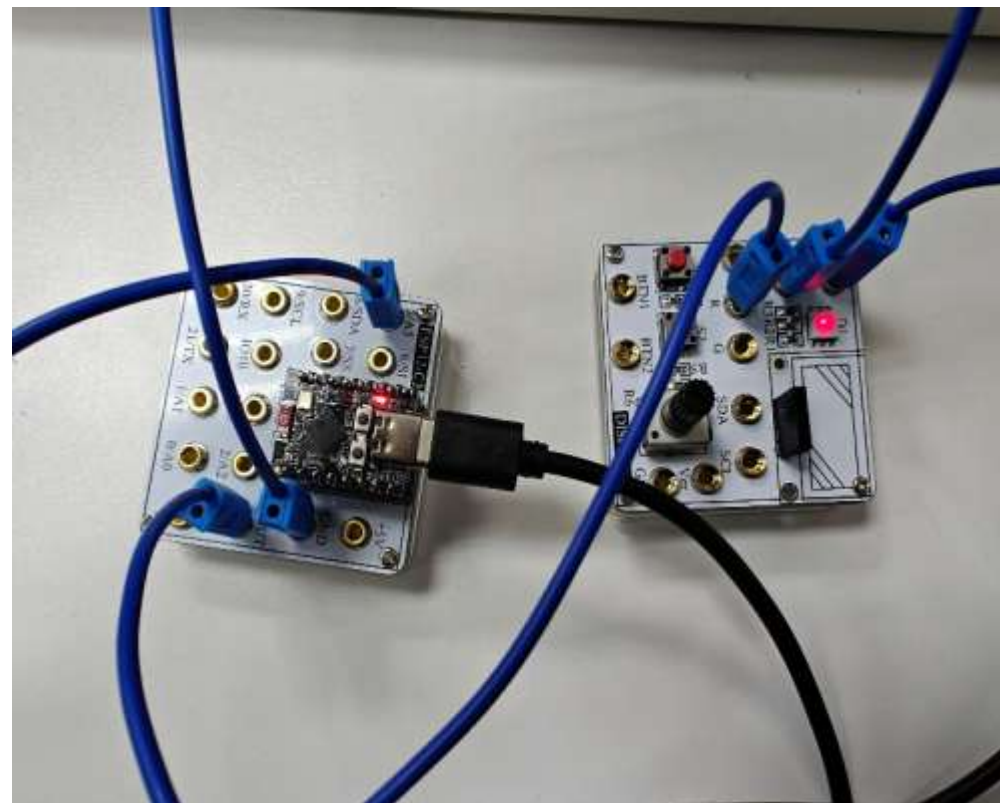




实验3：实物连线

■ LED

- 连接两个板子的3.3V脚
- 连接两个板子的接地GND
- 功能板的R引脚连接ESP32的5号引脚
- LED亮度随时间变化（呼吸灯）





实验3：实验代码

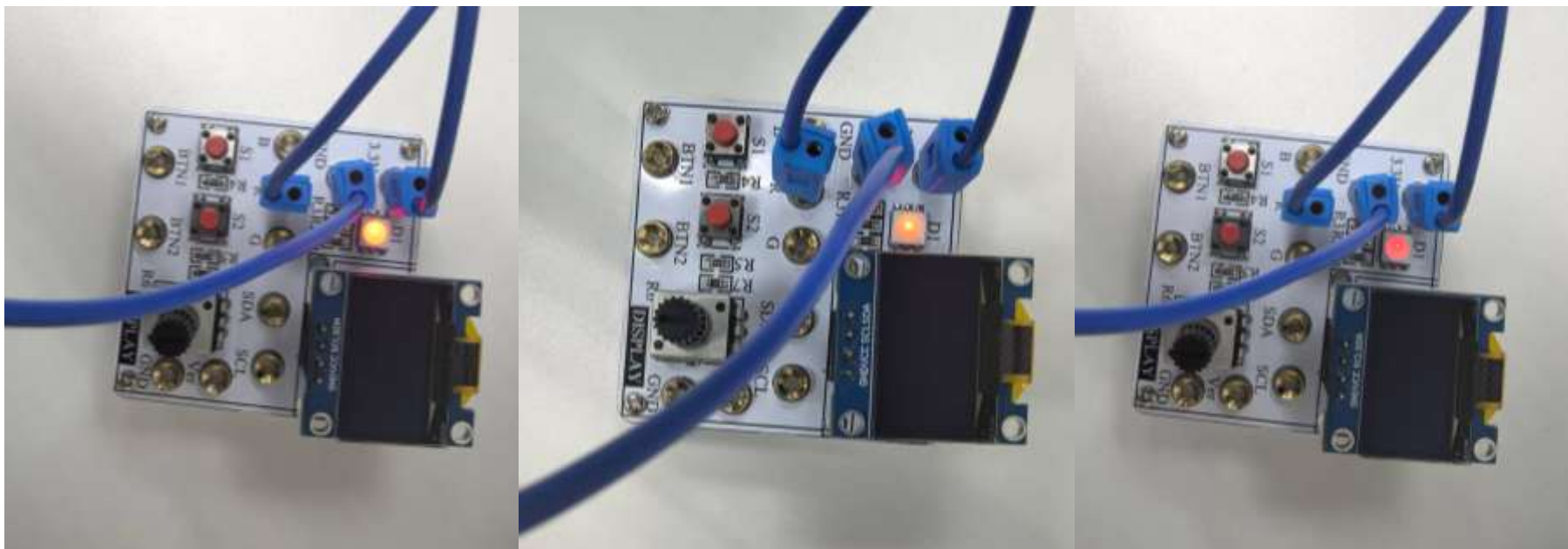
- `pinMode(pin,mode)` 设置引脚功能：读/写
- `analogWrite(pin,value)` 写（输出）引脚

```
1  #define led_red 5
2  int i = 0;
3  void setup() {
4      pinMode(led_red, OUTPUT);
5
6  }
7
8  void loop() {
9      for (i = 0; i <=510; i++){
10         if (i < 255)
11             analogWrite(led_red, i);
12         else
13             analogWrite(led_red, 510 - i);
14         delay(10);
15     }
16     delay(10);
17 }
18
```




实验3现象：输出电压高低变化

■ 对实验现象进行拍照记录





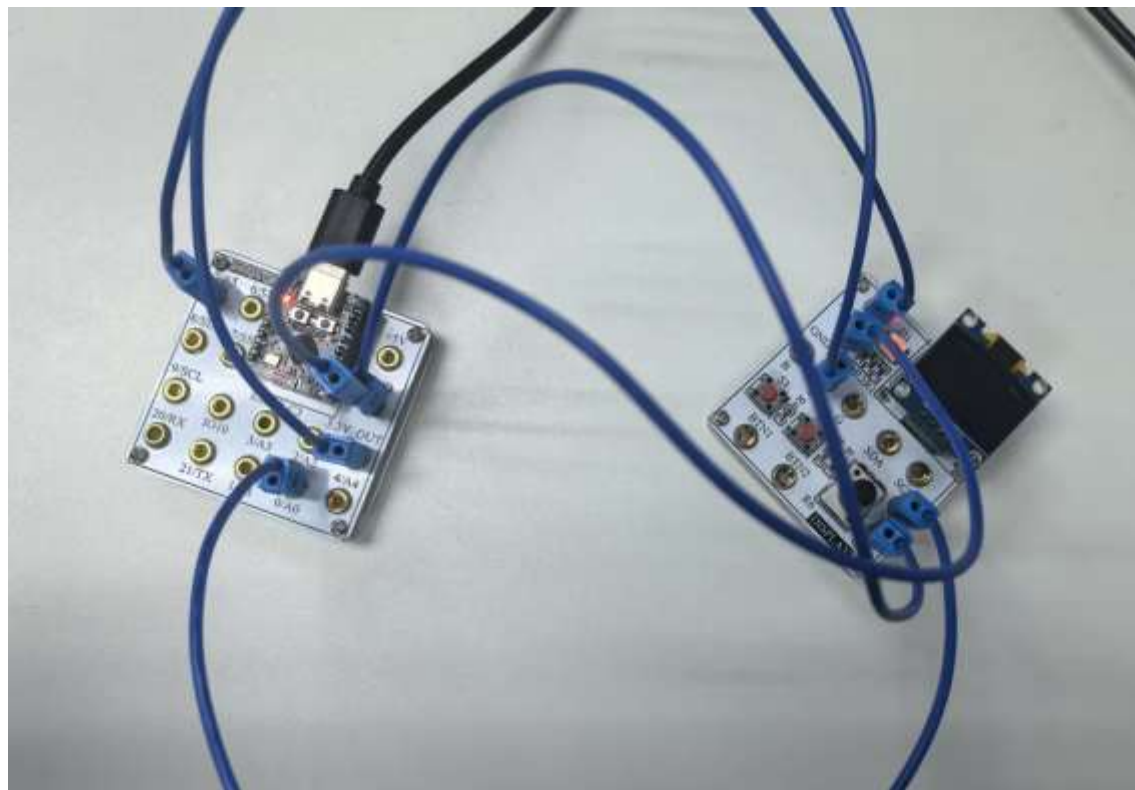
Arduino模拟I/O口实验

- 模拟信号与数字信号
- 模数(AD)-数模(DA)转换
- 实验1： 电位器模拟输入
- 实验2： PWM模拟输出
- 实验3： 呼吸灯电路
- **实验4： 亮度可控LED灯**
- **实验5： 颜色可调三色灯**



实验4：亮度可控LED灯

- 通过旋转电位器控制LED灯的亮度；
 - 模拟IO输入口接1个电位器（滑动变阻器）
 - 模拟IO输出口接LED灯，由电位器来控制LED灯的亮度

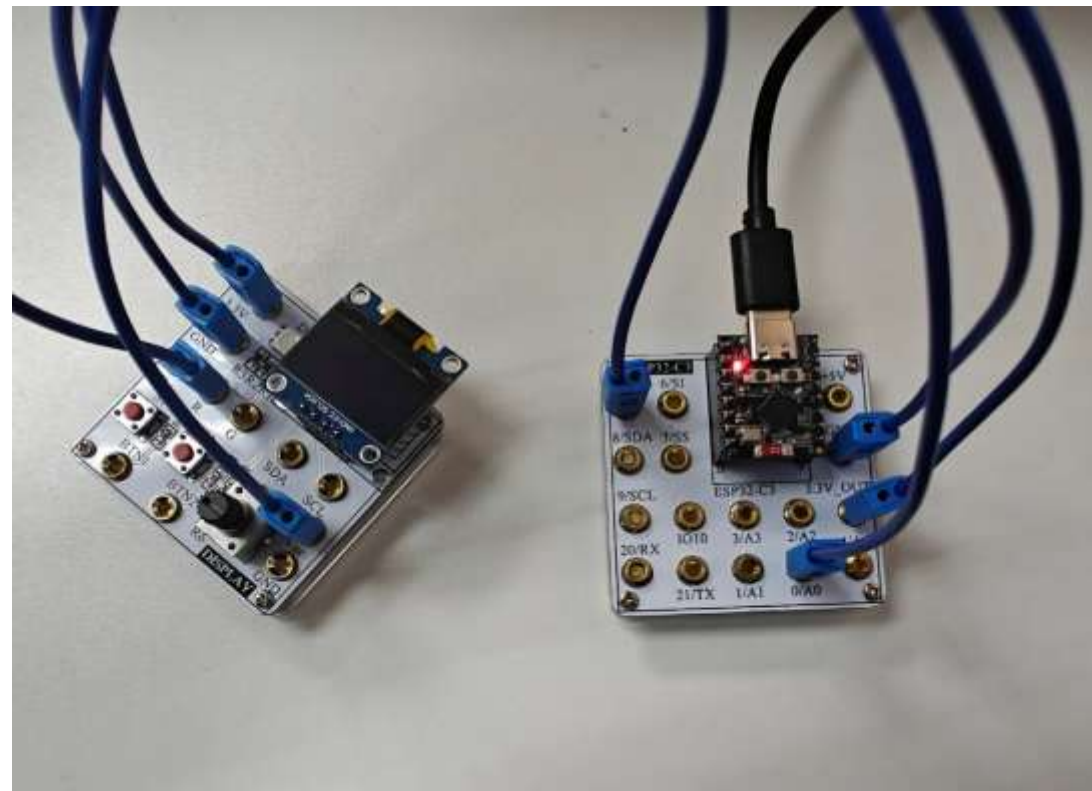




实验4：实验连线

■ 亮度可控LED灯

- 连接两个板子的3.3V
- 连接两个板子的接地GND
- 三色灯的R引脚连接ESP32的5号引脚
- 电位器Ver脚连接ESP32的0号引脚





实验4：实验代码

- **analogRead(pin,value)** 读（输入）电位器电压
- **analogWrite(pin,value)** 写（输出）LED引脚

```
1  #define pot 0
2  #define led 5
3  void setup() {
4      Serial.begin(9600); // 设置波特率
5  }
6
7  void loop() {
8      int potValue = analogRead(pot); // 读取电位器Ver端电压，量化为0~3.3-->0~4095
9      potValue = potValue / 16; // analogWrite量化范围0~255，与0~409516倍关系
10     analogWrite(led, potValue);
11     Serial.print("potValue = ");
12     Serial.println(potValue);
13     delay(100);
14 }
```



实验4：结果分析

- 记录5组串口监视器（截图）对应的输出和LED灯的亮度（拍照），并根据串口监视器输出的结果计算电位器两端电压
- `analogWrite(pin, value)`中参数value越大（0~255）输出的PWM波占空比越大吗？串口显示的数值与灯的亮度关系是什么？为什么？



Arduino模拟I/O口实验

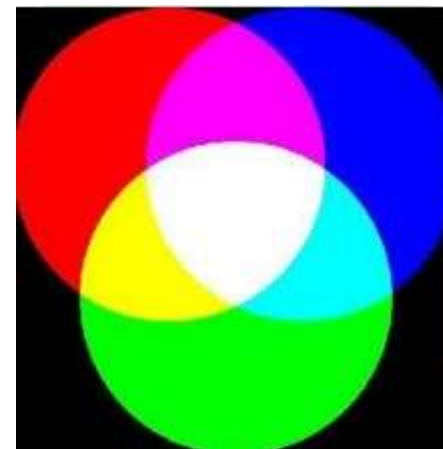
- 模拟信号与数字信号
- 模数(AD)-数模(DA)转换
- 实验1： 电位器模拟输入
- 实验2： PWM模拟输出
- 实验3： 呼吸灯电路
- 实验4： 亮度可控LED灯
- **实验5： 颜色可调三色灯**



实验5：颜色可调三色灯

■三色灯原理

- 三基色组合七色光原理：红+绿=黄 绿+蓝=青
红+蓝=紫 红+绿+蓝=白，可以由3种基色组合出来“红黄绿青蓝紫白”7色光
- 3种基色亮度不同还可以组成其他颜色的光，通过改变每种光亮度来实现颜色的渐变
- 通过**模拟电压**输出控制三色灯亮度，可以由3种基色组合出来其他颜色





实验5：颜色可调三色灯

- 通过旋转电位器，控制三色灯显示的组合色的变化
- 用示波器观测电位器输出模拟信号以及三色灯信号的时域波形
- 通过按键选择三色灯的颜色变化模式

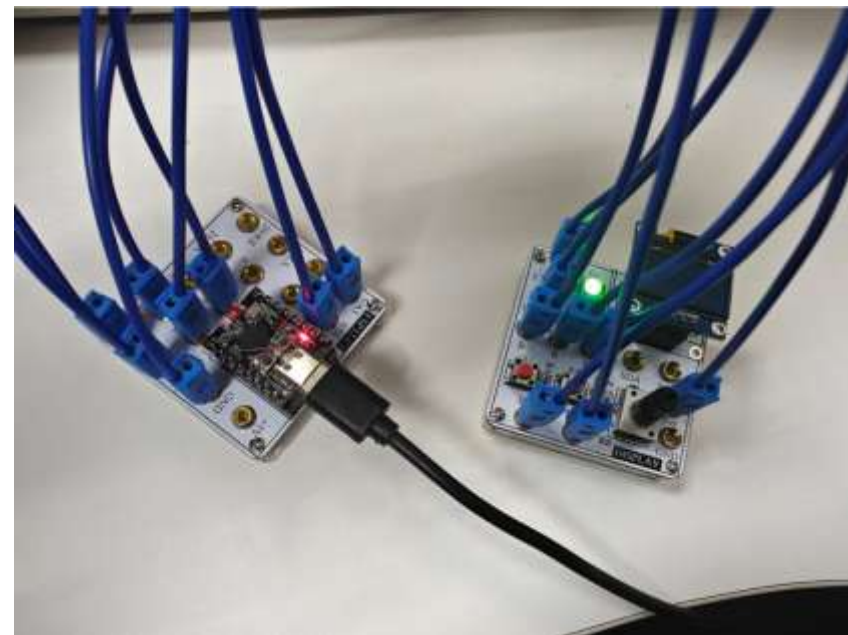




实验5：实物连线

■ 连接三色灯电路

- ESP32的**2、3、4号**口分别连接三色灯的**R、G、B** 引脚；
- 用**0号**口连接电位器Ver端，**电位器控制**三色灯的颜色；
- 用**5、6号**口分别连接**按键BTN1，BTN2**控制三色灯的变化模式；
- **三色灯R引脚和功能板Ver端**分别连接两条示波器**红色端**，两条线的**黑色端与板子GND**相连





实验5：实验代码

■ 定义引脚与按键

```
1  #define led_red 2
2  #define led_green 3
3  #define led_blue 4
4  #define btn1 5
5  #define btn2 6
6  #define ldrPin 0
7
8  int btn = 1;
9  int Oscii_obser_sign = 1; //示波器观察实验时置1，否则置0
10
11 void setup() {
12     pinMode(led_red, OUTPUT);
13     pinMode(led_green, OUTPUT);
14     pinMode(led_blue, OUTPUT);
15     pinMode(btn1, INPUT);
16     pinMode(btn2, INPUT);
17     Serial.begin(9600);
18 }
```



实验5：实验代码

- **digitalRead(pin) 读（输入）按钮**
- 定义**按钮BTN1**功能为三色灯交替显示红绿蓝（RGB）三基色，可通过旋转电位器调整三色灯的亮度
- **Oscii_obser_sign=1**
 - 示波器观察波形实验
- **Oscii_obser_sign=0**
 - 按钮选择三色灯的颜色变化模式实验

```
20 void loop() {
21     int potVal = analogRead(ldrPin);
22     Serial.println("Value = ");
23     Serial.println(potVal);
24     potVal = potVal / 16;
25     if(digitalRead(btn1) == 0)
26     {
27         btn = 1;
28     }
29     if(digitalRead(btn2) == 0)
30     {
31         btn = 0;
32     }
33     if(btn == 1)
34     {
35         if(Oscii_obser_sign == 1)
36         {
37             analogWrite(led_red, potVal);
38             analogWrite(led_green, 255);
39             analogWrite(led_blue, 255);
40             delay(500);
41         }
42         else
43         {
44             analogWrite(led_red, 255 - potVal);
45             analogWrite(led_green, 255);
46             analogWrite(led_blue, 255);
47             delay(500);
48             analogWrite(led_red, 255);
49             analogWrite(led_green, 255 - potVal);
50             analogWrite(led_blue, 255);
51             delay(500);
52             analogWrite(led_red, 255);
53             analogWrite(led_green, 255);
54             analogWrite(led_blue, 255 - potVal);
55             delay(500);
56         }
57     }
}
```




实验5：实验代码

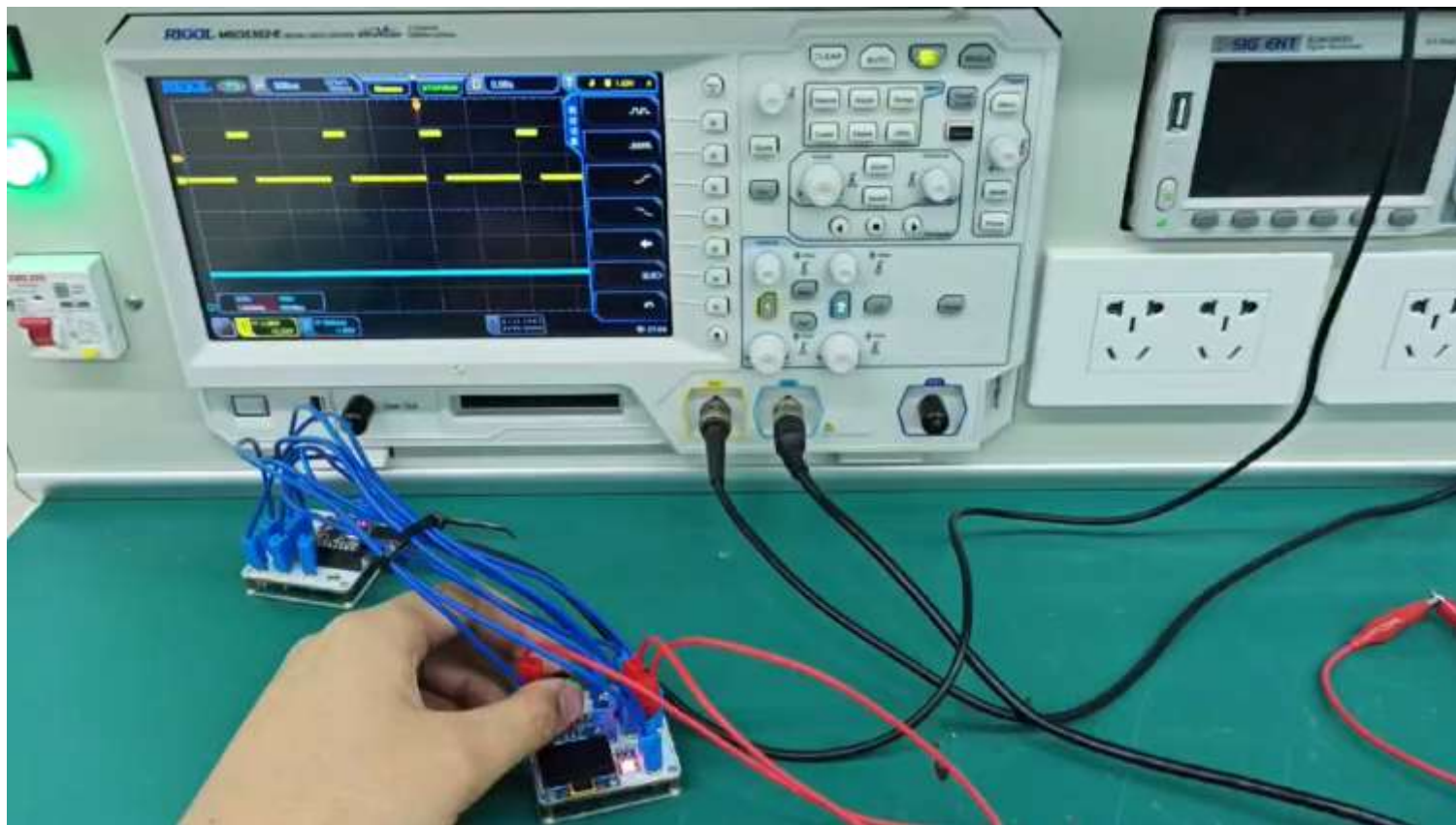
- 定义**按键BTN2**功能为三色灯颜色逐渐变化
- 可以用**电位器**控制三色灯的亮度

```
58     if(btn == 0)
59     {
60         for(int i = potVal; i > 0; i--)
61         {
62             analogWrite(led_red, i);
63             analogWrite(led_green, i);
64             analogWrite(led_blue, potVal - i);
65             delay(10);
66             if(digitalRead(btn1) == 0)break;
67         }
68         for(int i = 0; i < potVal; i++)
69         {
70             analogWrite(led_red, i);
71             analogWrite(led_green, i);
72             analogWrite(led_blue, potVal - i);
73             delay(10);
74             if(digitalRead(btn1) == 0)break;
75         }
76     }
77 }
```



实验5：示波器观测实验

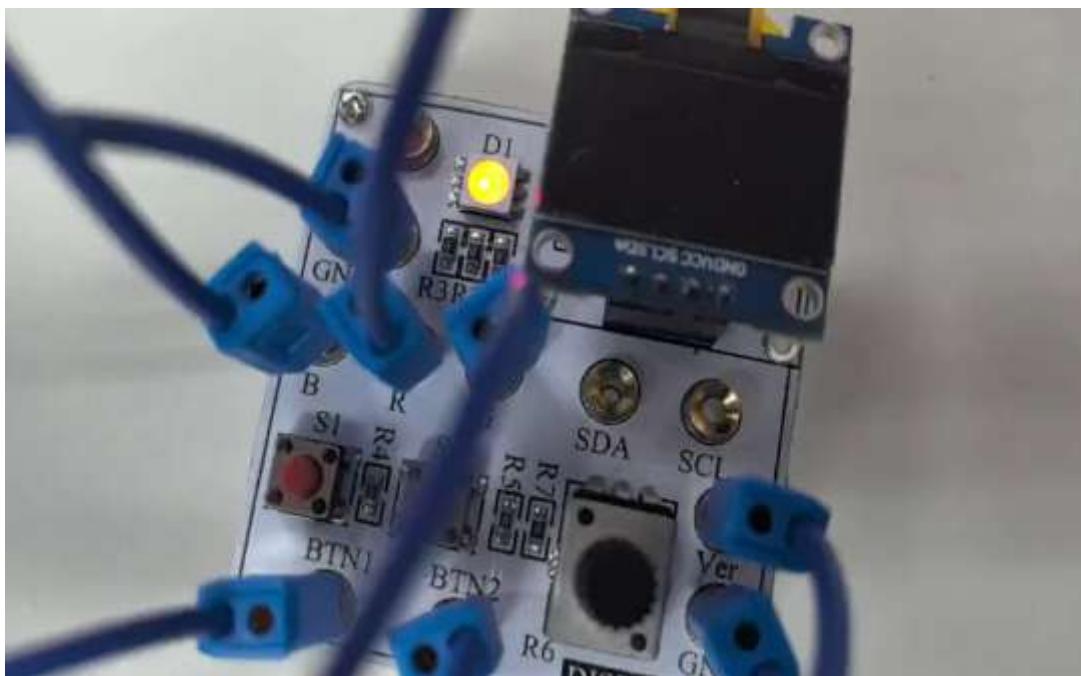
- 用示波器观测电位器输出模拟信号以及三色灯信号的时域波形
- 模拟信号与时域信号之间的关系是什么？为什么？





实验5：按键功能切换实验

BTN1功能



BTN2功能

